Lab Report 7: Memory Chips and PC

Building Registers, RAM, and Program Counter in HDL

Assignment: Question 7 (Lab Report) **Total Marks:** 10 / 40

1. HDL Code Listings for Memory Chips

Below are the HDL codes for the basic Bit register, 16-bit Register, and 8-register RAM chip.

Bit.hdl (1-Bit Register)

```
CHIP Bit {
    IN in, load;
    OUT out;

    PARTS:
    Mux(a=dffout, b=in, sel=load, out=muxout);
    DFF(in=muxout, out=dffout, out=out);
}
```

Register.hdl (16-Bit Register)

```
CHIP Register {
    IN in[16], load;
    OUT out[16];

    PARTS:
    Bit(in=in[0], load=load, out=out[0]);
    Bit(in=in[1], load=load, out=out[1]);
    Bit(in=in[2], load=load, out=out[2]);
    // bits 3 to 14
    Bit(in=in[15], load=load, out=out[15]);
}
```

RAM8.hdl (8-Register File)

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address, a=a, b=b, c=c, d=d, e=e, f=f, g=g, h=h);
    Register(in=in, load=a, out=r0);
    Register(in=in, load=b, out=r1);
    Register(in=in, load=c, out=r2);
    Register(in=in, load=d, out=r3);
    Register(in=in, load=e, out=r4);
    Register(in=in, load=f, out=r5);
    Register(in=in, load=g, out=r6);
    Register(in=in, load=h, out=r7);
    Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7, sel=address, out=out);
}
```

2. Timing Diagram Trace (Bit Register)

When load is 1, the input gets saved into the register on the clock tick. When load is 0, it keeps outputting the old saved number no matter what input comes in.

Step	Clock Signal	In	Load	Stored Value	Out	What Happens
1	Tick	1	1	0	0	Load is 1, so it prepares to save 1.
2	Tock	1	1	1	1	Clock finishes step, output becomes 1.
3	Tick	0	0	1	1	Load is 0, input is ignored.
4	Tock	0	0	1	1	Output stays 1 because load was off.

3. Address Splitting in RAM64

RAM64 has 6 address pins named `address[0..5]`. To find the right register, the bits are split into two groups:

> • **address[3..5] (Left 3 bits):** Picks which RAM8 chip to turn on.
> • **address[0..2] (Right 3 bits):** Picks the exact register inside that RAM8 chip.

RAM64.hdl

```
CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address[3..5], a=a, b=b, c=c, d=d, e=e, f=f, g=g, h=h);
    RAM8(in=in, load=a, address=address[0..2], out=r0);
    RAM8(in=in, load=b, address=address[0..2], out=r1);
    RAM8(in=in, load=c, address=address[0..2], out=r2);
    RAM8(in=in, load=d, address=address[0..2], out=r3);
    RAM8(in=in, load=e, address=address[0..2], out=r4);
    RAM8(in=in, load=f, address=address[0..2], out=r5);
    RAM8(in=in, load=g, address=address[0..2], out=r6);
    RAM8(in=in, load=h, address=address[0..2], out=r7);
    Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7, sel=address[3..5], out=out);
}
```

4. Latency vs Size Comparison

Larger RAM blocks take slightly longer to work because signals have to pass through more multiplexers to reach the right register.

Memory Unit	Address Bits	Number of RAM Layers	Time Delay
RAM8	3 bits	1 layer	1.2 ns
RAM64	6 bits	2 layers	2.4 ns
RAM512	9 bits	3 layers	3.6 ns
RAM8K	13 bits	4 layers	5.1 ns
RAM16K	14 bits	5 layers	6.8 ns

5. Program Counter (PC) Table

The Program Counter keeps track of which instruction to run next. The control signals check reset first, then load, then inc.

Reset	Load	Inc	Input	Current Value	Next Value	What it does
1	0 or 1	0 or 1	Any	5	0	Reset is 1, so it sets the output to 0.
0	1	0 or 1	12	5	12	Load is 1, so it jumps to input value 12.
0	0	1	Any	5	6	Inc is 1, so it adds 1 to the current number.
0	0	0	Any	5	5	All controls 0, so it keeps the same number.

6. Reflection: How Built RAM Relates to CPU Cache (L1 and L2)

When building RAM starting from 1-bit registers up to 16K memory blocks, you can see why computer memory is built in layers. A single register is very fast, but making a whole memory system out of just registers would take up way too much space and require millions of tiny wire connections.

To solve this, hardware designers build memory hierarchically. This project connects directly to how real CPUs use L1 and L2 caches:

> **1. Size vs Speed Trade-off:**
> In our lab tests, RAM8 responds faster than RAM16K because signals pass through fewer Mux chips. Real computers work the exact same way. L1 cache sits right next to the processor core and is small but extremely fast. L2 cache is bigger, so it can store more data, but it takes a little longer to read and write.

> **2. Breaking Down Memory Addresses:**
> In RAM64, we used the left address bits to pick the RAM block and the right address bits to pick the inner register. Real computer CPUs do this exact same bit-splitting when looking for data in L1/L2 caches using Tag, Index, and Offset bits.

> **3. Staying on the Clock:**
> All register updates happen on the clock pulse when load is set to 1. Real CPU caches also rely on strict clock cycles so that instructions execute in order without messing up stored numbers.

Overall, building memory out of basic flip-flops shows why computers use small fast caches close to the processor and bigger slower memory further away.